

Graph-Aware Deep Reinforcement Learning for Adaptive Capacity Planning in Distributed Personalized Regenerative Medicine Manufacturing Networks

Zhaowei Li^{a,*}

^aH. Milton Stewart School of Industrial and Systems Engineering, Georgia Institute of Technology, Atlanta, GA, USA

Abstract

Distributed make-to-order manufacturing networks require adaptive decision policies because demand, supply availability, and resource states evolve over time and across connected facilities. This study develops a graph-aware deep reinforcement learning framework for adaptive capacity planning in distributed personalized regenerative medicine manufacturing networks. The artificial intelligence contribution is the integration of graph convolutional networks with deep deterministic policy gradient learning. The graph convolutional component encodes inter-facility dependencies and transshipment relationships, while the actor-critic reinforcement learning component learns continuous capacity, inventory, and transshipment decisions through interaction with a production simulation environment. The engineering application is autologous cell therapy manufacturing, where production can begin only when a patient specimen, an idle bioreactor, and sufficient con-

*Corresponding author.

Email address: zli686@gatech.edu (Zhaowei Li)

sumables are available. Computational experiments evaluate the proposed framework against myopic, one-period look-ahead, two-period look-ahead, and isolated-facility heuristic policies under supply disruption and demand forecast uncertainty. The results indicate that the proposed GCN-DDPG framework achieves robust cost performance under high disruption rates and adapts effectively to inaccurate and non-stationary demand forecasts. The study contributes an intelligent, network-aware decision-support approach for distributed healthcare manufacturing and provides a generalizable modeling framework for engineering systems with sequential decisions and graph-structured dependencies.

Keywords: Graph convolutional networks, Deep reinforcement learning, Capacity planning, Distributed manufacturing, Personalized regenerative medicine, Transshipment

1. Introduction

Distributed make-to-order manufacturing networks are increasingly important in engineering systems where products are individualized, production capacity is limited, and service outcomes depend on time-sensitive operational decisions. Personalized regenerative medicine (PRM), including autologous cell therapy, is a representative example. In this setting, manufacturing cannot begin until a patient specimen, an idle bioreactor, and sufficient consumables are simultaneously available. Because products are patient-specific and cannot be produced in advance, the system faces high demand uncertainty, limited opportunities for pooling inventory, and substantial risk from disruptions in the supply of critical materials.

12 Distributed and point-of-care manufacturing can reduce fulfillment time
 13 by locating production capacity closer to patients, but it also creates a more
 14 complex engineering decision problem. Facilities are not independent: short-
 15 ages at one clinic can sometimes be mitigated through inventory transship-
 16 ment, specimen reassignment, or capacity relocation from connected facili-
 17 ties. These interactions create spatial dependencies across the manufactur-
 18 ing network and require policies that coordinate decisions at the network
 19 level rather than optimizing each facility in isolation. Prior work has shown
 20 the value of simulation and analytical models for comparing centralized and
 21 point-of-care autologous cell therapy supply chains [1] and for decentral-
 22 ized capacity planning under uncertainty [2, 3]. However, many existing
 23 approaches rely on simplified transition models, deterministic approxima-
 24 tions, or heuristic look-ahead rules, which can limit their adaptability when
 25 demand and supply conditions change.

26 The artificial intelligence contribution of this study is a graph-aware deep
 27 reinforcement learning framework that integrates graph convolutional rep-
 28 resentation learning with actor-critic policy optimization. The engineer-
 29 ing application is adaptive capacity planning, inventory replenishment, and
 30 transshipment decision-making in distributed PRM manufacturing networks.
 31 Reinforcement learning is appropriate because the problem is sequential:
 32 decisions made in one epoch affect future inventories, resource availability,
 33 waiting specimens, and service outcomes. Graph convolutional networks are
 34 appropriate because the state of the system is naturally relational: each man-
 35 ufacturing facility is a node, and transshipment or coordination links define
 36 how local decisions propagate through the network.

37 This paper develops a graph convolutional network and deep determinis-
38 tic policy gradient framework for intelligent decision support in distributed
39 make-to-order manufacturing. The graph convolutional component encodes
40 facility-level states and neighboring-facility information into network-aware
41 embeddings. The deep deterministic policy gradient component then learns
42 continuous capacity, inventory, and transshipment actions from interactions
43 with a production simulation environment. The resulting framework is de-
44 signed to learn adaptive policies without requiring exact prior knowledge of
45 demand distributions or supplier disruption processes.

46 The main contributions of this study are as follows:

- 47 • We formulate distributed PRM capacity planning as a sequential en-
48 gineering decision problem involving capacity adjustment, consumable
49 inventory replenishment, and network-level transshipment under de-
50 mand and supply uncertainty.
- 51 • We develop a graph-aware deep reinforcement learning framework that
52 combines graph convolutional state embedding with deep deterministic
53 policy gradient learning.
- 54 • We show how graph-based state representations allow the decision-
55 maker to use both local facility conditions and neighboring-facility in-
56 formation when learning coordinated policies.
- 57 • We evaluate the proposed framework against myopic, look-ahead, and
58 isolated-facility heuristics under supply disruption and demand forecast
59 error scenarios.

- 60 • We discuss the implications of the framework for intelligent decision
61 support in distributed, make-to-order healthcare manufacturing net-
62 works and related engineering systems.

63 The remainder of this paper is organized as follows. Section 2 describes
64 the distributed PRM capacity planning problem. Section 3 reviews related
65 work. Section 4 presents the proposed graph-aware deep reinforcement learn-
66 ing framework. Section 5 describes the computational experiments and re-
67 sults. Section 6 concludes the study and identifies future research directions.

68 **2. Distributed Network Manufacturing and Capacity Planning Prob-**
69 **lem**

Sets and parameters	
Γ	Planning horizon indexed by t
T	Number of time periods required to manufacture one of the PRM products, $T < \Gamma$
N	Set of manufacturing facility indexed by n
m_t^n	The number of products that begin manufacturing at facility n at epoch t
d_t^n	Number of demand arrive at facility n in epoch t
State variables	
s_t^n	Number of specimens waiting for manufacturing at facility n at epoch t
r_t^n	Number of available reagents at facility n at epoch t
$\mathbf{b}_t^n$	Vector $(b_t^{n,1}, b_t^{n,2}, \dots, b_t^{n,T})$, where $b_t^{n,\tau}$ is the number of bioreactors that requires
P_t^n	more days to be completed at facility n at epoch t
	Maximum number of reagents that can be purchased by facility n for next epoch $t + 1$
Decision variables	
w_t^n	Number of specimens added to or subtracted from facility n at epoch t and is received or removed before epoch $t + 1$
e_t^n	Number of reagents added to or subtracted from facility n at epoch t and is received or removed before epoch $t + 1$
q_t^n	Number of idle bioreactors added to or subtracted from facility n at epoch t and is received or removed before epoch $t + 1$
p_t^n	Vector of the quantity of reagents that will be purchased by facility n for next epoch $t + 1$

71 In this section, we formulate the capacity planning problem for distributed
72 manufacturing systems as a production simulation model based on Markov

73 Decision Process (MDP). Suppose the distributed manufacturing system con-
74 sists of L manufacturing facilities that produce one kind of make-to-order
75 PRM product. Each manufacturing facility has its production capacity, e.g.,
76 bioreactors and reagent inventory, and can purchase reagents from source
77 material suppliers. During manufacturing, facilities receive patients' speci-
78 mens, and each facility produces the PRM products based on the number of
79 bioreactors and reagents at hand. Thus, we assume inventory transshipment,
80 capacity relocation, and demand sharing are allowed among facilities. Also,
81 demand and supplier disruptions are assumed to be stochastic processes.
82 This implies the supply can be less than or higher than demands at different
83 epochs. A two dimensional example ($L = 2$) is illustrated in Figure 1. The
84 notations are summarized in Table 1 .

85 For the N facilities, T epochs of manufacturing time are required to
86 produce one of the PRM products, and the planning horizon consists of
87 Γ epochs where $\Gamma > T$.

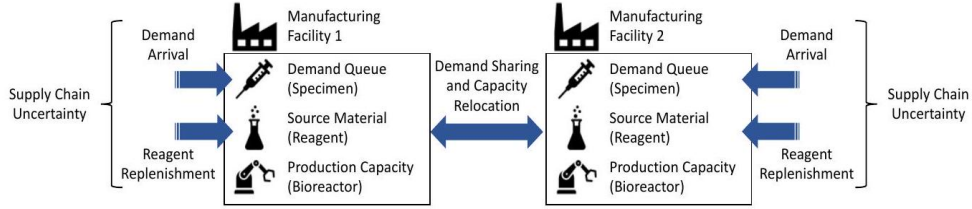


Figure 1: Illustration of a distributed PRM manufacturing system with two facilities.

88 At epoch $t \in \{1, 2, \dots, \Gamma\}$, the production status of n -th facility for
89 $n = 1, \dots, N$ is expressed as a state vector $\mathbf{x}_t^n$, where

$$\mathbf{x}_t^n = (s_t^n, r_t^n, \mathbf{b}_t^n, P_t^n), \quad (1)$$

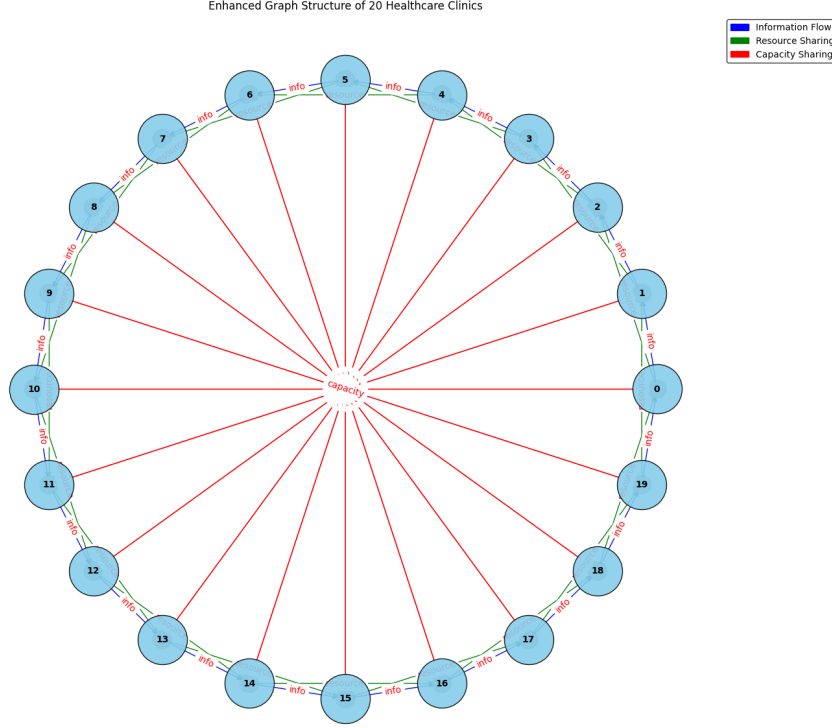


Figure 2: Enhanced Graph Structure of 20 Healthcare Clinics.

Based on the state variables at epoch t , the facilities need to take action to adjust their production capacity for epoch $t + 1$. We define the action $\mathbf{a}_t^n$ as a vector that consists of the amount of capacity sharing and reagent replenishment. The action is decided at the beginning of the production period t and executed before the start of $t + 1$.

$$\mathbf{a}_t^n = (w_t^n, e_t^n, q_t^n, p_t^n), \quad (2)$$

The goal of capacity planning is to minimize total production cost by dynamically selecting the actions from Equation (2) for all facilities, denoted

by $\mathbf{A}_t = (\mathbf{a}_t^1, \dots, \mathbf{a}_t^N)$, for each epoch $t \in \Gamma$. We define $V(\mathbf{X}_t)$ as the total future cost of the selecting actions $\mathbf{A}_t$ when the states from all facilities, denoted by $\mathbf{X}_t = (\mathbf{x}_t^1, \dots, \mathbf{x}_t^N)$, are observed at epoch $t \in \Gamma$. The objective function can be expressed as a dynamic programming problem:

$$V(\mathbf{X}_t) = \min_{\mathbf{A}_t} E[c(\mathbf{A}_t; \mathbf{X}_t) + \gamma V(\mathbf{X}_{t+1})], \quad (3)$$

which is useful for finding the optimal action $\mathbf{A}_t$ for each state $\mathbf{X}_t$ at each epoch $t \in \Gamma$, where $c(\mathbf{A}_t; \mathbf{X}_t)$ is the immediate cost at time t , and $\gamma \in [0, 1]$ is a discount factor for balancing the impact of future costs in deciding actions. The choice of γ depends on the user's goal; if γ is closer to 1 (to 0), then it means the user cares more about long (short) term cost; if $\gamma = 0$, then only the cost for current epoch is considered. Equation (3) generalizes the optimality equation for the capacity planning problem in Li(2021) [2] in that cost functions $\mathbf{c}$ and $\mathbf{V}$ and the cumulative distribution function for the expectation operator can be unknown. In the next section we will propose a DRL approach based on the DDPG algorithm for determining the solution of Equation (3) by learning the cost functions $\mathbf{c}$ and $\mathbf{V}$ and the cumulative distribution function for the expectation operator using data-driven approaches.

3. Literature Review

3.1. Personalized regenerative medicine supply chains

PRM supply chains differ from conventional make-to-stock manufacturing systems because products are individualized, time-sensitive, and tightly coupled with patient-specific clinical pathways. In autologous cell therapy,

119 for example, patient material must be collected, transported, manufactured,
 120 quality-tested, and returned for administration. These requirements make
 121 fulfillment time, service reliability, and manufacturing capacity utilization
 122 central performance measures. Simulation-based studies have examined how
 123 centralized and point-of-care manufacturing strategies affect cost, fulfillment
 124 time, service level, and resource utilization in autologous cell therapy sup-
 125 ply chains [1]. Related work has also studied decentralized capacity plan-
 126 ning and resilience in autologous cell therapy manufacturing networks using
 127 approximate dynamic programming and transshipment-based coordination
 128 mechanisms [2].

129 *3.2. Reinforcement learning for supply chain and manufacturing decisions*

130 Reinforcement learning has been applied to a range of supply chain and
 131 manufacturing problems. Early work used reinforcement learning to deter-
 132 mine inventory replenishment policies in multi-echelon supply chains [4] and
 133 perishable inventory systems [5]. Subsequent studies incorporated deep rein-
 134 forcement learning to handle higher-dimensional state spaces and simulation-
 135 based decision environments [6, 7, 8]. Policy-gradient and actor-critic meth-
 136 ods have also been used for capacitated supply chain optimization, retail
 137 inventory replenishment, safety stock optimization, and joint pricing and in-
 138 ventory control [9, 10, 11, 12]. These studies demonstrate the potential of
 139 reinforcement learning for sequential operational decision-making, but many
 140 do not jointly address capacity adjustment, inventory replenishment, trans-
 141 shipment, and demand sharing in distributed make-to-order manufacturing
 142 systems.

143 3.3. *Deep reinforcement learning for regenerative medicine capacity planning*

144 Recent work by Tseng et al. [3] applied deep reinforcement learning
145 to dynamic capacity planning in decentralized regenerative medicine sup-
146 ply chains. That work provides an important foundation by showing that
147 learned policies can adapt to demand variation in PRM manufacturing sys-
148 tems. The present study builds on this line of research by adding a graph
149 representation layer to explicitly encode the network structure among dis-
150 tributed facilities. This extension is important because transshipment and
151 capacity-sharing decisions create inter-facility dependencies that are difficult
152 to represent with flat vectorized states alone.

153 3.4. *Graph representation learning for networked decision systems*

154 Many engineering systems, including transportation networks, supply
155 chains, energy systems, and distributed manufacturing networks, are nat-
156 urally represented as graphs. In such settings, node-level decisions are influ-
157 enced by neighboring nodes, edge costs, and the broader network topology.
158 Graph convolutional networks provide a mechanism for aggregating local and
159 neighborhood information into learned state embeddings. For distributed
160 PRM manufacturing, graph representation learning is particularly relevant
161 because each facility’s capacity, inventory, and waiting demand should be in-
162 terpreted in relation to connected facilities and transshipment opportunities.

163 3.5. *Research gap*

164 Existing PRM capacity planning studies provide valuable analytical and
165 simulation-based insights, and recent deep reinforcement learning work demon-
166 strates the feasibility of data-driven policy learning. However, limited work

has integrated graph representation learning with reinforcement learning for distributed PRM capacity planning. Without an explicit graph encoder, reinforcement learning policies may fail to fully exploit the spatial dependencies induced by inter-facility transshipment and capacity sharing. This study addresses this gap by developing a GCN-DDPG framework that combines network-aware state embedding with adaptive continuous-action policy learning.

4. Graph-Aware Deep Reinforcement Learning Framework

Integrating DDPG with GCN for Distributed Manufacturing (DDPG-GCN):

Introduction to the Method

distributed manufacturing networks for Personalized Regenerative Medicine (PRM) involve complex, interconnected facilities whose operating states are best represented as graphs rather than flat vectors. Classical reinforcement learning (RL) models, which treat each facility as independent, struggle to capture these spatial dependencies. To address this, we integrate the Deep Deterministic Policy Gradient (DDPG) algorithm with a Graph Convolutional Network (GCN), enabling the agent to learn joint capacity and transshipment decisions while accounting for network structure.

Rationale for the DDPG-GCN Integration

The DDPG algorithm is well suited for continuous control problems, providing a model-free, off-policy actor–critic mechanism for policy learning.

189 However, DDPG alone assumes vectorized inputs and cannot directly lever-
 190 age the relational information among facilities. Conversely, GCNs excel at
 191 extracting structural embeddings from graph-structured data but lack tem-
 192 poral learning capabilities. Their integration allows the model to encode
 193 spatial correlations through GCN layers and temporal decision dynamics
 194 through DDPG updates.

195 Formally, let $G_t = (V, E_t)$ represent the manufacturing network at epoch
 196 t , where each node $v_i \in V$ denotes a facility with state features x_i^t and edge
 197 weights e_{ij}^t indicate logistic or transshipment costs. The GCN encodes local
 198 and neighborhood information using a layer-wise propagation rule:

$$h_i^{(l+1)} = \sigma\left(W^{(l)} \sum_{j \in \mathcal{N}(i)} \frac{1}{c_{ij}} h_j^{(l)}\right), \quad (4)$$

199 where $h_i^{(0)} = x_i^t$, $W^{(l)}$ are trainable weights, $\sigma(\cdot)$ is a ReLU activation, and
 200 c_{ij} is a degree-based normalization factor. The resulting embedding $H_t =$
 201 $\{h_i^{(L)}\}_{i=1}^N$ captures the current global state of the network.

202 *Overview of the DDPG-GCN Workflow*

203 The method initiates with a dual initialization: the DDPG’s actor and
 204 critic networks and the GCN for graph state processing. Given an observed
 205 state X_t , represented as raw graph data reflecting the current status of the
 206 manufacturing network, the GCN is used as a preprocessing unit. It trans-
 207 forms this raw graph into a condensed representation X'_t , which retains the
 208 critical structural information of the original graph.

209 Following this transformation, the DDPG algorithm gets involved. Using
 210 the graph-processed state X'_t , the actor-network selects an action, which is

211 based on the actor–critic architecture. This action could range from decisions
 212 regarding resource allocation to inventory replenishments. Once executed
 213 within the manufacturing environment, the system provides a feedback loop
 214 in the form of a reward, essentially evaluating the decision’s efficacy.

215 The DDPG then uses this feedback, in tandem with the replay buffer,
 216 to refine both the actor and critic networks, ensuring better decision-making
 217 in subsequent iterations. The GCN, on its part, continuously updates its
 218 understanding of the graph structure as the manufacturing network evolves,
 219 ensuring it remains adaptive and accurate in its representations.

220 *Initialization*

- 221 • Critic network $Q(X_t, A_t|\theta_Q)$ initialized with weights θ_Q .
- 222 • Actor network $\mu(X_t|\theta_\mu)$ initialized with weights θ_μ .
- 223 • GCN initialized with weights ω for processing graph states.

224 *DDPG Component*

225 The DDPG agent operates on the GCN-encoded state H_t to generate
 226 an action vector $A_t = \{\mathbf{a}_i^t\}_{i=1}^N$, where $\mathbf{a}_i^t$ specifies the capacity adjustment,
 227 inventory replenishment, and transshipment quantities for facility i . The
 228 actor network $\mu(H_t|\theta_\mu)$ outputs continuous actions, while the critic network
 229 $Q(H_t, A_t|\theta_Q)$ estimates the expected cumulative cost. The optimization fol-

230 lows the standard DDPG updates:

$$y_i = c_i(A_i; H_i) + \gamma Q'(H_{i+1}, \mu'(H_{i+1}|\theta_{\mu'})|\theta_{Q'}), \quad (5)$$

$$L_Q = \frac{1}{N} \sum_i (y_i - Q(H_i, A_i|\theta_Q))^2, \quad (6)$$

$$\nabla_{\theta_\mu} J = \frac{1}{N} \sum_i \nabla_a Q(H_i, a|\theta_Q)|_{a=\mu(H_i)} \nabla_{\theta_\mu} \mu(H_i|\theta_\mu). \quad (7)$$

231 *Reward Function*

232 The reward (negative cost) encourages policies that balance cost mini-
233 mization and service-level stability. For each epoch t , the immediate reward
234 is defined as

$$r_t = -\left(C_t^{\text{reagent}} + C_t^{\text{bio}} + C_t^{\text{trans}}\right), \quad (8)$$

235 where each term corresponds to the reagent, bioreactor, and transshipment
236 cost components introduced in Eq. (14). This formulation allows the agent
237 to learn trade-offs between resource utilization and timely delivery.

238 **GCN Encoder:** Each facility node i aggregates features from its neighbors
239 $\mathcal{N}(i)$ using layer-wise message passing:

$$h_i^{(l+1)} = \sigma \left(W^{(l)} \sum_{j \in \mathcal{N}(i)} h_j^{(l)} \right),$$

240 where $W^{(l)}$ are trainable weights and $\sigma(\cdot)$ is an activation function. This
241 produces graph-embedded representations used by the actor-critic networks
242 for coordinated decision-making.

243 *Anticipated Outcomes and Benefits*

244 The DDPG-GCN model offers several tangible benefits:

Algorithm 1 DDPG–GCN for Distributed PRM Capacity Planning

Require: Discount factor γ , soft-update rate τ , actor and critic learning rates α_μ, α_Q , exploration scale σ , minibatch size N .

- 1: Initialize GCN encoder g_ω , actor $\mu(\cdot \mid \theta_\mu)$, critic $Q(\cdot, \cdot \mid \theta_Q)$, target networks μ' and Q' , and replay buffer $\mathcal{R}$.
 - 2: **for** each environment step t **do**
 - 3: Observe raw graph state X_t and compute graph embedding $Z_t = g_\omega(X_t)$.
 - 4: Select exploratory action $A_t = \mu(Z_t \mid \theta_\mu) + \epsilon_t$, where $\epsilon_t \sim \mathcal{N}(0, \sigma^2 I)$.
 - 5: Project or clip A_t to the feasible set and execute the resulting action $\hat{A}_t$.
 - 6: Observe cost/reward r_t , next state X_{t+1} , and store $(X_t, \hat{A}_t, r_t, X_{t+1})$ in $\mathcal{R}$.
 - 7: **if** an update is scheduled **then**
 - 8: Sample minibatch $(X_i, A_i, r_i, X_{i+1})_{i=1}^N$ from $\mathcal{R}$ and compute $Z_i = g_\omega(X_i)$, $Z_{i+1} = g_\omega(X_{i+1})$.
 - 9: Compute targets $y_i = r_i + \gamma Q'(Z_{i+1}, \mu'(Z_{i+1} \mid \theta_{\mu'}) \mid \theta_{Q'})$.
 - 10: Update critic by minimizing $L_Q = N^{-1} \sum_i (y_i - Q(Z_i, A_i \mid \theta_Q))^2$.
 - 11: Update actor using the deterministic policy gradient $\nabla_{\theta_\mu} J$.
 - 12: Backpropagate through the GCN encoder and softly update target networks.
 - 13: **end if**
 - 14: **end for**
 - 15: **return** trained actor $\mu(g_\omega(X) \mid \theta_\mu)$.
-

- 245 • **Enhanced Prediction and Analysis:** By leveraging the relation-
246 ships between clinics, our GCN framework can generate predictions
247 with greater accuracy and nuanced analysis, surpassing the capabili-
248 ties of models that consider clinics in isolation.
- 249 • **Pattern Identification:** The GCN is adept at detecting trends and
250 patterns that may extend across multiple clinics, thereby uncovering
251 insights that isolated analyses could overlook.
- 252 • **Efficient Graph Processing:** By harnessing the power of GCNs, the
253 model can effectively handle states that are naturally represented as
254 graphs, ensuring no critical information is lost in translation.
- 255 • **Optimized Decision-Making:** The DDPG component ensures that
256 the actions selected are both contextually relevant (thanks to the GCN-
257 processed states) and optimal in terms of the overall system objectives.
- 258 • **Adaptive Learning:** As the manufacturing network evolves, both
259 the DDPG and GCN components continuously refine their parame-
260 ters, ensuring the model remains adaptive and responsive to real-world
261 changes.

262 In essence, the integration offers a solution that is greater than the sum
263 of its parts, paving the way for more efficient, resilient, and optimized dis-
264 tributed manufacturing systems.

265 This proposed DDPG-GCN model offers a robust approach for distributed
266 manufacturing. By melding the strengths of GCN and DDPG, it provides

267 a nuanced understanding of intricate manufacturing networks, ensuring ef-
268 ficient and optimized decision-making. The dynamic nature and intricate
269 dependencies found in distributed manufacturing necessitate advanced so-
270 lutions capable of understanding and optimizing such networks. The inte-
271 gration of DDPG and GCN, as presented in this paper, provides a promis-
272 ing avenue for addressing the challenges inherent in distributed manufactur-
273 ing systems. With the capability to process graph-based states effectively,
274 the DDPG-GCN model offers substantial improvements in service level, effi-
275 ciency, and resilience against disruptions.

276 5. Computational Experiments

277 This section presents two case studies designed to evaluate the efficacy of
278 the proposed DDPG-GCN framework in a distributed PRM manufacturing
279 system. The case studies aim to illustrate the framework’s adaptability to
280 varying demand forecast accuracies and its capacity for making informed
281 decisions within a complex network of interconnected clinics.

282 5.1. Objective

283 The objective of this study is to compare the performance of the pro-
284 posed GCN-DDPG (Graph Convolutional Network - Deep Deterministic Pol-
285 icy Gradient) framework with traditional heuristic methods in the context of
286 distributed manufacturing.

287 5.2. Methodology

288 5.2.1. Benchmarking Setup

289 To conduct a fair comparison, we implement the following traditional
290 heuristic methods within the same simulation environment used for the GCN-
291 DDPG framework:

- 292 • **MYO (Myopic):** This heuristic focuses on minimizing single-period
293 costs without considering future demand.
- 294 • **MDL-1 (One-period Mean Demand Lookahead):** MDL-1 ex-
295 tends the foresight to one lookahead period, using deterministic state
296 transitions based on mean demand.
- 297 • **MDL-2 (Two-period Mean Demand Lookahead):** MDL-2 ex-
298 tends the foresight to two lookahead periods, still relying on determin-
299 istic state transitions based on mean demand.
- 300 • **ISO (Isolated Regional Facilities):** ISO assumes no resource shar-
301 ing between facilities and focuses solely on reagent replenishment.

302 5.2.2. Performance Metrics

303 We compare the methods based on the following key performance metrics:

- 304 • **Cost Efficiency:** Evaluate the cost efficiency of each method in terms
305 of the overall expenses incurred while satisfying 95% service level.
- 306 • **Responsiveness to Demand and Supply Variability:** Assess how
307 each method adapts to changes in demand and supply variability in
308 dynamic scenarios.

309 5.2.3. Heuristic Implementations

310 We provide brief descriptions of the heuristic methods implemented:

- 311 • **MYO (Myopic):** MYO focuses on minimizing costs within the cur-
312 rent period without considering future consequences.
- 313 • **MDL-1 and MDL-2:** These methods extend the planning horizon to
314 one and two lookahead periods, respectively, using deterministic state
315 transitions based on mean demand.
- 316 • **ISO (Isolated Regional Facilities):** ISO assumes no sharing of re-
317 sources between manufacturing facilities and primarily concentrates on
318 reagent replenishment.

319 5.2.4. GCN-DDPG Implementation

320 The model is implemented in `Python 3.10` using `PyTorch`. The GCN
321 encoder contains two convolutional layers with 64 and 32 hidden units, re-
322 spectively, followed by ReLU activations. The DDPG actor and critic each
323 consist of three fully connected layers with dimensions (256, 128, 64). All
324 parameters are initialized with Xavier initialization. Training uses the Adam
325 optimizer with learning rates of 1×10^{-4} for the actor and 1×10^{-3} for the
326 critic. The replay buffer size is 10^6 , minibatch size is 256, discount factor
327 $\gamma = 0.99$, and target update rate $\tau = 0.005$. Exploration noise follows a
328 decaying Ornstein–Uhlenbeck process with initial $\sigma = 0.2$. Each model is
329 trained for 1.0×10^6 steps and evaluated on 500 Monte Carlo test runs to
330 ensure robustness. All experiments are conducted on an NVIDIA RTX 4090
331 GPU.

332 This implementation ensures reproducibility and stability while maintain-
 333 ing scalability across larger network configurations.

334 5.3. distributed manufacturing simulation

Algorithm 2 State normalization

Require: State variables: s_t^n , r_t^n , b_t^n , and p_t^n

1: Maximal values of the state variables for all facilities: M_s , M_r , M_b , and M_p

Ensure: Normalized state variables: $\hat{s}_t^n$, $\hat{r}_t^n$, $\hat{b}_t^n$, $\hat{p}_t^n$

2: **for** $n = 1$ to N **do**

3: $\hat{s}_t^n = s_t^n / M_s$

4: $\hat{r}_t^n = r_t^n / M_r$

5: $\hat{b}_t^n = b_t^n / M_b$

6: $\hat{p}_t^n = p_t^n / M_p$

7: **end for**

335 To evaluate the proposed method, we now construct a simulation en-
 336 vironment of a decentralized manufacturing system consisting of $N = 20$
 337 manufacturing facilities and its dedicated reagent supplier. Each PRM prod-
 338 uct requires $T = 3$ epochs of manufacturing time, and the considered finite
 339 problem horizon contains $\Gamma = 52$ decision epochs. For every epoch t , there
 340 will be d_t^n new demand for facility n . The demand arrival process for facility
 341 n follows a Poisson distribution $\text{Poisson}(\lambda_n)$. The supplier disruption follows
 342 independent Bernoulli processes $\text{Ber}(\zeta_n)$, i.e., the facility n has a probability
 343 ζ_n to have supplier disruption and could not purchase any reagent for the
 344 epoch. This simulation can be further adjusted to reflect any real situation,

e.g., supply disruption and demand uncertainty, that may be encountered. Such flexibility in using the simulation system will be demonstrated in Section 5.4 when we compare GCN-DDPG with other methods.

The epoch is assumed to occur weekly, and the transshipment of specimens, bioreactors, and reagents at epoch t arrive before epoch $t+1$. After the demand and source materials have arrived, the next observing state can be expressed by $\mathbf{x}_{t+1}^n = (s_{t+1}^n, r_{t+1}^n, \mathbf{b}^n, P_{t+1})$ for facility $n = 1, \dots, N$, according to definition Equation (1). The state variables are updated by the following rules:

$$\begin{aligned} s_{t+1}^n &= s_t^n - m_t^n + d_t^n + w_t^n \\ r_{t+1}^n &= r_t^n - m_t^n + e_t^n + p_t^n \\ b_{t+1}^{n,\tau} &= \begin{cases} b_t^{n,0} - m_t^n + b_t^{n,1} + q_t^n & \text{if } \tau = 0 \\ b_t^{n,\tau+1} & \text{if } 1 \leq \tau \leq T-2 \\ m_t^n & \text{if } \tau = T-1 \end{cases} \end{aligned}$$

where $m_t^n = \min(s_t^n, b_t^{n,0}, r_t^n)$ is the number of products that begin manufacturing at epoch t , d_t^n is the demand arriving at facility n in epoch t , and P_{t+1} follows an exogenous Markov chain from P_t . The single-period cost in period t of all facility n can be written as:

$$\begin{aligned} \mathbf{C}(\mathbf{A}_t; \mathbf{x}_t) &= \sum_{j=t}^{\Gamma} \sum_{n=1}^N \mathbf{C}^n(\mathbf{w}_t^n, \mathbf{e}_t^n, \mathbf{q}_t^n, \mathbf{p}_t^n; \mathbf{s}_t^n) \\ \mathbf{C}^n(\mathbf{w}_t^n, \mathbf{e}_t^n, \mathbf{q}_t^n, \mathbf{p}_t^n | \mathbf{s}_t^n) &= C_{\text{Reg},t}^n + C_{\text{Bio},t}^n + C_{\text{Tran},t}^n \end{aligned}$$

where the meanings of $C_{\text{Reg},t}^n$, $C_{\text{Bio},t}^n$, and $C_{\text{Tran},t}^n$ are summarized below.

- 359 • $C_{\text{Reg},t}^n = c_R p_t^n + h_R (r_{t+1}^n - s_{t+1}^n)^+ + p_R (s_{t+1}^n - r_{t+1}^n)^+$ is the cost related
 360 to reagent at epoch t , including the reagent replenishment cost $c_R p_t^n$,
 361 the reagent overstock holding cost $h_R (r_{t+1}^n - s_{t+1}^n)^+$, and the reagent
 362 understock penalty $p_R (s_{t+1}^n - r_{t+1}^n)^+$;
- 363 • $C_{\text{Bio},t}^n = h_B (b_{t+1}^{n,0} - s_{t+1}^n)^+ + p_B (s_{t+1}^n - b_{t+1}^{n,0})^+$ is the cost related to
 364 bioreactor at epoch t , including the bioreactor overstock holding cost
 365 $h_B (b_{t+1}^{n,0} - s_{t+1}^n)^+$, and the bioreactor understock penalty $p_B (s_{t+1}^n - b_{t+1}^{n,0})^+$;
- 366 • $C_{\text{Tran},t}^n = K_R |e_t^n| + K_B |q_t^n| + K_S |w_t^n|$ is the total transshipping cost at
 367 epoch t , including the cost of transshipping e_t^n units of reagent from/to
 368 facility n , the cost of relocating q_t^n bioreactors from/to facility n , and
 369 the cost of transshipping w_t^n specimens from facility i to facility n .

370 5.4. Case Study 1: Interconnected Clinic Network Analysis

371 In this case study, we assume the demand forecast for heuristics is accu-
 372 rate and compare the cost performance of GCN-DDPG with the heuristics
 373 under three levels of supply disruption rate. This case study aims to demon-
 374 strate that GCN-DDPG can learn effective policies under different uncer-
 375 tainty scenarios. Figure 3 illustrates the average total cost of GCN-DDPG
 376 and the heuristics under 0.05, 0.3 and 0.6 supplier disruption rates. The re-
 377 sult of MYO is omitted in the figure due to its poor performance. The result
 378 shows that GCN-DDPG can achieve comparable cost performance with the
 379 heuristics when the supplier disruption rate is low, e.g., 0.05 and 0.3. When
 380 the supplier disruption rate is high, e.g., 0.6, GCN-DDPG outperforms all
 381 the heuristics. The cost differences between GCN-DDPG and heuristics are

382 calculated by $(Cost_{Heuristic} - Cost_{GCN-DDPG})/Cost_{Heuristic} \times 100$ and listed
 383 in Table 1.

384 5.4.1. Results

385 The results indicate that the GCN-DDPG framework can leverage network-
 386 level information to improve resource allocation and respond to demand
 changes more effectively.

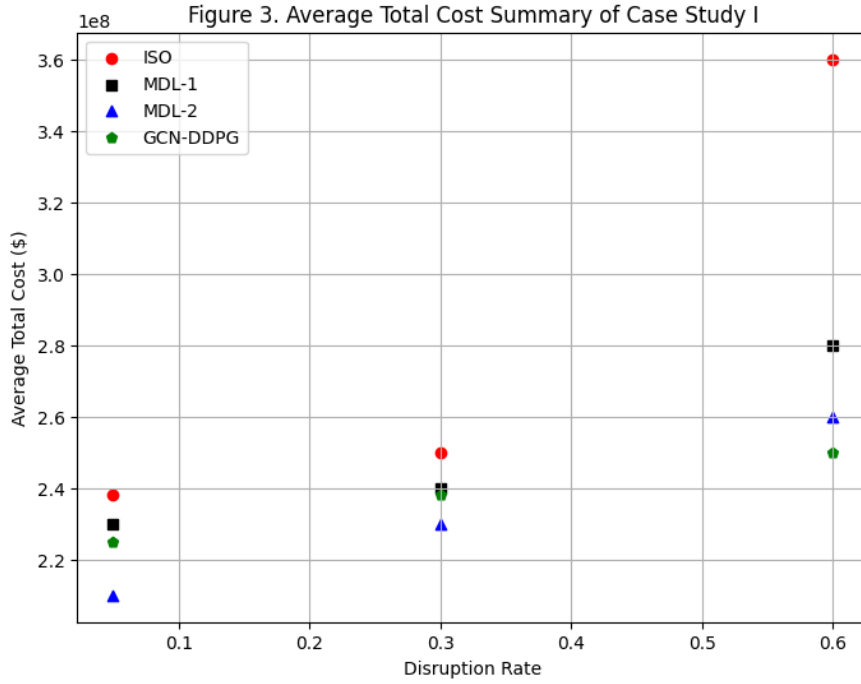


Figure 3: Average total cost summary of case study 1.

388 5.4.2. Analysis:

389 In the comparative analysis of disruption management strategies, the
 390 GCN-DDPG method emerges as the most robust, demonstrating unparal-
 391 leled cost-efficiency across varying disruption rates. Unlike other evaluated

Table 1: Cost performance comparison of case study I (% increase)

Method	Disruption Rate		
	0.05	0.3	0.6
ISO	5.46%	4.80%	30.56%
MDL-1	2.17%	0.83%	10.71%
MDL-2	-7.14%	-3.48%	3.85%
GCN-DDPG	0.00%	0.00%	0.00%

392 methods, GCN-DDPG maintains stable costs, suggesting its superior ability
 393 to mitigate the financial impact of disruptions. The ISO approach, while ef-
 394 fective at lower disruption levels, incurs a significant cost increase of 30.56%
 395 at higher disruption rates, highlighting its limitations under strain. MDL-1
 396 and MDL-2 show varied performances, with MDL-2 offering cost reductions
 397 at lower disruptions. However, neither can match GCN-DDPG’s consistent
 398 cost performance, which shows no increase and suggests an adaptive, resilient
 399 approach. This consistent performance of GCN-DDPG underscores its po-
 400 tential as a scalable solution for healthcare networks, ensuring operational
 401 reliability and financial prudence even in the face of escalating disruptions.

402 5.5. Case Study 2: Adaptability to Non-Stationary Demand

403 In this case study, we assume the demand forecast is less accurate for
 404 heuristic methods. Instead, we use a range of demand levels in the simula-
 405 tion model for training our GCN-DDPG approach, as opposed to the actual
 406 demand distribution. This case study aims to demonstrate that GCN-DDPG
 407 can learn a single policy that can adapt to different uncertainty scenarios,
 408 mimicking a real-world application.

409 In the computer experiment, we let the heuristics generate capacity plans
 410 using a demand distribution $\text{Poisson}(d)$ and $d = \frac{250}{52}$. We then set the
 411 actual demand distribution as $\text{Poisson}(d')$, where $d' = d + \epsilon \cdot d$ and $\epsilon \in$
 412 $\{-0.2, 0, 0.2, 0.3, 0.4, 0.5\}$, to generate data for testing heuristic methods.
 413 This approach tests the adaptability of our GCN-DDPG model to demand
 414 forecast errors that are $-20\%, 0\%, 20\%, 30\%, 40\%, 50\%$. In addition to the
 415 constant error, we also consider scenarios of non-stationary errors where the
 416 demand forecast error changes over time as listed in Table 2. The average
 417 total cost of GCN-DDPG and the heuristics under different levels of demand
 418 forecasting accuracy are summarized in Figure 4.

419 *5.5.1. Results*

420 The results indicate that the GCN-DDPG framework can leverage network-
 421 level information to improve resource allocation and respond to demand
 422 changes more effectively.

423 We train the GCN-DDPG model to make effective capacity planning de-
 424 cisions under different uncertainty scenarios. The capacity planning model
 425 for the GCN-DDPG approach utilizes a uniform parameter $\text{Uniform}(d, 2d)$
 426 for each time step, ensuring adaptability to the varying demand patterns.

427 *5.5.2. Analysis*

428 The results of our experiment show that the GCN-DDPG framework can
 429 outperform all heuristic methods in cases of $40\%, 50\%$, and non-stationary
 430 demand forecast error. This demonstrates the robustness of the GCN-DDPG
 431 approach in adapting to a wide range of demand uncertainties, offering a
 432 promising solution for distributed manufacturing in the face of unpredictable

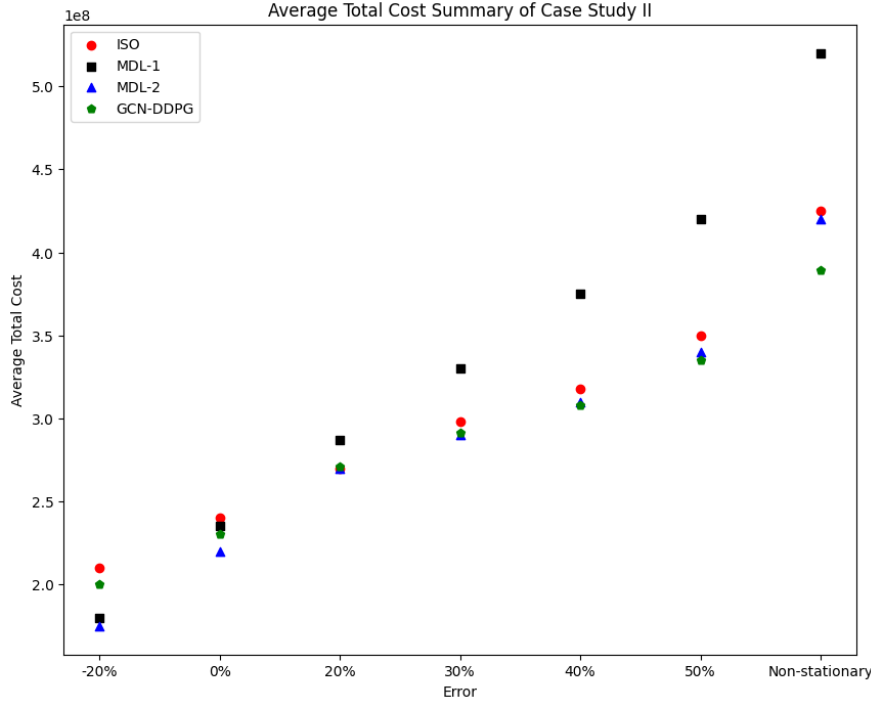


Figure 4: Average total cost summary of case study 2.

433 demand.

434 6. Discussion and Conclusions

435 The two case studies conducted provide a comprehensive comparison be-
 436 tween the cost performance of the proposed GCN-DDPG framework and
 437 four ADP-based methods under conditions where demand distributions are
 438 known a priori. The findings affirm that GCN-DDPG can generate effective
 439 actions and policies, demonstrating performance on par with ADP-based
 440 methods across various uncertainty scenarios. In the second case study, we
 441 explored scenarios where the actual demand distribution is unknown, training
 442 a single GCN-DDPG policy to address this uncertainty. The results endorse

Table 2: Summary of non-stationary demand forecast error design for Case Study II across 20 clinics.

Epoch	Error Rate Across Clinics
1-13	Varies from 30% to 60%, alternating between clinics
14-26	Varies from 10% to 80%, alternating between clinics
27-39	Varies from 50% to 100%, alternating between clinics
40-52	Varies from 40% to 60%, alternating between clinics

the framework’s capability to learn and control capacity planning effectively under diverse demand uncertainty scenarios, thereby highlighting its applicability to real-world problems where system uncertainties are unknown.

While the capacity planning problem we explored is inspired by PRM manufacturing, the flexibility of the GCN-DDPG approach suggests its potential applicability across a wide range of manufacturing applications to enhance supply chain resilience, even without a predefined system structure.

The GCN-DDPG framework, leveraging the power of simulation and the learning capabilities of Deep Reinforcement Learning, is particularly well-suited for modeling complex systems and learning optimal control policies. This approach offers significant benefits for PRM supply chains, including the ability to make immediate decisions using a neural network-based trained policy, avoiding the need for time-consuming optimization processes typical of analytical methods. Furthermore, a well-trained GCN-DDPG policy can be adapted to different system settings, such as varying levels of demand uncertainty, without the necessity for retraining.

However, challenges remain in the deployment of GCN-DDPG within PRM supply chains. Achieving acceptable performance with DRL policies

461 may require meticulous hyperparameter tuning or extended training times.
462 The simulation environment must accurately reflect the supply chain’s state
463 to allow for efficient retraining and maintenance of policies. Significant
464 changes in the system may necessitate policy retraining to ensure the qual-
465 ity of decisions. Practitioners must develop a robust workflow for training,
466 maintaining, and validating GCN-DDPG policies to mitigate the risk of sub-
467 optimal decision-making.

468 **Future Work** This study motivates several potential research extensions.
469 First, the simulation model can be improved by adding variables such as pa-
470 tient health condition, raw material quality, product quality, process failure
471 rate, machine breakdown process, and operator availability. Furthermore,
472 other PRM supply chain activities such as product distribution and prepa-
473 ration, patient treatment activities, and production scheduling processes can
474 be added to the simulation.

475 **Data Availability**

476 The data used in this study are generated from a simulation model.
477 The simulation parameters and experimental settings are described in the
478 manuscript. Code and additional materials are available from the corre-
479 sponding author upon reasonable request.

480 **Declaration of Competing Interest**

481 The author declares that there is no known competing financial interest or
482 personal relationship that could have appeared to influence the work reported
483 in this paper.

484 References

- 485 [1] K. Wang et al. “A simulation-based comparison of centralized and
486 point-of-care supply chain strategies for autologous cell therapy”. In:
487 *Cytotherapy* 25.12 (Dec. 2023). Epub 2023 Sep 22, pp. 1370–1379. DOI:
488 10.1016/j.jcyt.2023.08.007.
- 489 [2] Junxuan Li and Chelsea C. White. “Capacity planning in a decentral-
490 ized autologous cell therapy manufacturing network for low-cost re-
491 siliience”. In: *Flexible Services and Manufacturing Journal* 35.2 (2023),
492 pp. 295–319. ISSN: 1936-6590. DOI: 10.1007/s10696-022-09475-6.
493 URL: <https://doi.org/10.1007/s10696-022-09475-6>.
- 494 [3] Chin-Yuan Tseng et al. “Deep Reinforcement Learning Approach for
495 Dynamic Capacity Planning in Decentralized Regenerative Medical
496 Supply Chains”. In: *International Journal of Production Research* (2021).
497 Accepted for publication.
- 498 [4] Ilaria Giannoccaro and Pierpaolo Pontrandolfo. “Inventory manage-
499 ment in supply chains: a reinforcement learning approach”. In: *Interna-*
500 *tional Journal of Production Economics* 78.2 (2002), pp. 153–161. ISSN:
501 0925-5273. DOI: [https://doi.org/10.1016/S0925-5273\(00\)00156-](https://doi.org/10.1016/S0925-5273(00)00156-0)
502 [0](https://doi.org/10.1016/S0925-5273(00)00156-0). URL: [https://www.sciencedirect.com/science/article/pii/](https://www.sciencedirect.com/science/article/pii/S0925527300001560)
503 [S0925527300001560](https://www.sciencedirect.com/science/article/pii/S0925527300001560).
- 504 [5] Ahmet Kara and Ibrahim Dogan. “Reinforcement learning approaches
505 for specifying ordering policies of perishable inventory systems”. In:
506 *Expert Systems with Applications* 91 (2018), pp. 150–158. ISSN: 0957-
507 4174. DOI: <https://doi.org/10.1016/j.eswa.2017.08.046>.

- 508 URL: [https://www.sciencedirect.com/science/article/pii/](https://www.sciencedirect.com/science/article/pii/S0957417417305900)
509 [S0957417417305900](https://www.sciencedirect.com/science/article/pii/S0957417417305900).
- 510 [6] Volodymyr Mnih et al. “Playing Atari with Deep Reinforcement Learn-
511 ing”. In: *CoRR* abs/1312.5602 (2013). arXiv: 1312.5602. URL: <http://arxiv.org/abs/1312.5602>.
512
- 513 [7] Afshin Oroojlooy jadid et al. “A Deep Q-Network for the Beer Game:
514 Deep Reinforcement Learning for Inventory Optimization”. In: *Man-
515 ufacturing & Service Operations Management* 24 (Feb. 2021). DOI:
516 10.1287/msom.2020.0939.
- 517 [8] Hao Wang et al. “Dynamic inventory replenishment strategy for aerospace
518 manufacturing supply chain: combining reinforcement learning and multi-
519 agent simulation”. In: *International Journal of Production Research* 60
520 (Jan. 2022), pp. 1–20. DOI: 10.1080/00207543.2021.2020927.
- 521 [9] Zedong Peng et al. “Deep Reinforcement Learning Approach for Ca-
522 pacitated Supply Chain optimization under Demand Uncertainty”. In:
523 Nov. 2019, pp. 3512–3517. DOI: 10.1109/CAC48633.2019.8997498.
- 524 [10] Yassine Hachaïchi, Yassine Chemingui, and Mariem Affes. “A Pol-
525 icy Gradient Based Reinforcement Learning Method for Supply Chain
526 Management”. In: Dec. 2020, pp. 135–140. DOI: 10.1109/IC_ASET49463.
527 2020.9318258.
- 528 [11] Edward Kosasih and Alexandra Brintrup. “Reinforcement Learning
529 Provides a Flexible Approach for Realistic Supply Chain Safety Stock
530 Optimisation”. In: *IFAC-PapersOnLine* 55 (Oct. 2022), pp. 1539–1544.
531 DOI: 10.1016/j.ifacol.2022.09.609.

- 532 [12] Rui Wang et al. “Solving a Joint Pricing and Inventory Control Problem
533 for Perishables via Deep Reinforcement Learning”. In: *Complexity* 2021
534 (Jan. 2021), pp. 1–17. DOI: 10.1155/2021/6643131.